

UNIVERSITY OF PETROLEUM AND ENERGY STUDIES

School of Computer Science · Dehradun

CLOUD APPLICATION AND DEVELOPMENT

Large-Scale Hospital Event Analytics

Project Design Document

Project ID P35 · Data-Intensive Computing

SUBMITTED BY

Apratim Saxena

SAP ID 500119616
Batch B4 · CCVT
apratim.119616@stu.upes.ac.in

SUBMITTED TO

Prof. A Sai Rao

Cloud Application
and Development

Table of Contents

LARGE-SCALE HOSPITAL EVENT ANALYTICS · PROJECT P35

1. What we are building 3

2. Requirement traceability 3

3. System overview 4

4. Cloud architecture 5

5. Technology stack and justification 5

6. Database selection and justification 6

7. Data model 7

8. Synthetic dataset generation 8

9. The analytics pipeline 9

10. REST API design 10

11. Authentication and authorization 11

12. Logging and observability 12

13. Metrics 13

14. Load and scalability testing 14

15. Controlled failure scenario 14

16. Deployment 15

17. Demonstration plan 17

18. Risks and scope management 18

1. What we are building

A cloud-deployed batch analytics engine that turns a large stream of synthetic hospital events into three analytical products: admission patterns, resource utilisation, and unusual workload periods. In one line: **a data-intensive analytics pipeline over hospital event data, not a CRUD application.**

The system has three planes. An **ingest plane** accepts hospital events over REST and writes them immutably to raw storage. A **processing plane** picks up analytics jobs from a queue and runs them MapReduce-style across a pool of workers, writing aggregated results to separate result tables. A **query plane** serves those pre-computed results back over REST in milliseconds, regardless of how many million raw events sit underneath.

Why this satisfies cloud-specific behaviour

The allocation warns against a project that is only basic CRUD. Five properties of this design are cloud-specific and cannot be demonstrated by a CRUD application:

1. **Asynchronous decoupling.** A job request returns `202 Accepted` in under 50 ms; the work happens on workers the API server never addresses directly.
2. **Horizontal scaling.** Throughput is a function of worker-container count, not code. The same 1,000,000-event job completes faster on four workers than on one, with no change but a replica count.
3. **Failure tolerance through redelivery.** Killing a worker mid-job does not lose the job; the broker redelivers the unacknowledged task and the job still completes.
4. **Separation of hot and cold paths.** Raw data is append-only and large; results are small, read-hot and cached. Query latency stays flat as raw volume grows 100x.
5. **Stateless services behind shared state.** Every API container is interchangeable because all state lives in Postgres and Redis.

2. Requirement traceability

Every mandated requirement maps to one concrete, demonstrable artefact.

#	Requirement as stated	How we satisfy it	Evidence at evaluation
1	Core functionality: admission patterns, resource use, unusual workload periods	Three reducer jobs producing three result tables	Three query endpoints returning data
2	Minimum 5 meaningful REST endpoints	13 endpoints across auth, ingest, jobs, analytics, ops	Swagger UI at `/docs`
3	Proper data model for main entities	4 raw/dimension tables + 3 result tables + 1 job table	ER diagram, `\dt` in psql
4	Authentication/authorization	JWT for humans, API key for ingest service, 3-role RBAC	403 shown live on a wrong-role call
5	Deploy on cloud/deployable environment	Docker images on Render: API web service, Celery worker, managed Postgres, managed Key Value	Public HTTPS URL
6	Logs for important events, failures, actions	Structured JSON logs with correlation IDs at every stage	`docker logs` filtered by one correlation ID
7	At least 2 project-relevant metrics	Processing time, ingest throughput, p95 latency, cache hit ratio	`/api/v1/metrics` + result tables
8	Load testing at low/medium/high	Locust at 10, 100, 500 users against local Compose; 10 and 100 repeated in cloud	Four Locust HTML reports
9	Cloud architecture diagram	Architecture specified in section 4; diagram submitted separately	Architecture walk-through
10	API documentation + final report	Auto-generated OpenAPI + this document with results	Both submitted
11	Sufficiently large synthetic dataset	Generator producing 10k / 100k / 1M events	Row counts in psql

#	Requirement as stated	How we satisfy it	Evidence at evaluation
12	Batch/distributed/data-intensive processing	Celery map tasks + chord reducer, partitioned by day x department	Live job run
13	Store both raw input and processed output	`raw.*` (append-only) vs `analytics.*` (derived)	Two schemas shown
14	Process three dataset sizes, compare time	Benchmark over 10k/100k/1M, timings recorded	Comparison table
15	APIs to query analytical results	Three analytics endpoints with filters	Live curl
16	3+ meaningful cloud/architectural components	API service, Redis broker, Celery pool, Postgres, cache, object storage	Architecture walk-through
17	Controlled failure scenario + recovery	Kill a worker mid-job; broker redelivers; job completes	Live kill during demonstration
18	Git repo, 8+ meaningful commits	15 planned commits mapped to the 15-day plan	`git log --oneline`
19	Live modification readiness	Adding a fourth metric touches three files	Practised in advance

Three requirements are met at roughly twice the minimum because the extra cost is small: endpoint count (13 against 5), workload levels (a fourth breaking-point run), and metrics (four against two).

3. System overview

The system is best understood by following one event from generation to answer.

1. **Generation.** The generator produces an event: patient 48213 admitted to Cardiology on 14 March at 02:40, severity 4, via Emergency, bed C-212.
2. **Ingestion.** It is `POST`ed to `/api/v1/events` with an API key. The API validates it against a Pydantic schema, stamps `ingested_at` and a `correlation_id`, and inserts it into `raw.hospital_events`, returning `201 Created`. No analysis happens here; this path must stay fast. The row is append-only, partitioned by month, and never updated or deleted.
3. **Job trigger.** An analyst calls `POST /api/v1/jobs` for an admission-pattern analysis over March. The API writes a `queued` row to `analysis_jobs`, pushes a message to the broker, and returns `202 Accepted` in under 50 ms.
4. **Partition and map.** A coordinator divides the window into one partition per (day x department) pair; March across 8 departments gives 248. Workers pull partitions in parallel, each streaming only its slice with a server-side cursor so memory stays flat, and emit key-value pairs keyed on `(date, department)`.
5. **Shuffle and reduce.** Intermediate values go to Redis grouped by reducer key, a few hundred keys rather than millions of rows. Once every map task has acknowledged, a Celery **chord** fires the reducer, which folds them into daily counts, average length of stay, peak occupancy and rolling z-scores.
6. **Persist and query.** Results are written to `admission_metrics`, `resource_metrics` and `anomaly_windows`, and the job row flips to `completed`. `GET /api/v1/analytics/admissions` then reads the pre-computed rows, not the raw table: single-digit milliseconds, cached for 60 seconds.

The three planes

Plane	Components	Optimised for	Scales by
Ingest	FastAPI write endpoints, `raw.hospital_events`	Write throughput, never blocking	Adding API replicas
Process	Redis broker, Celery workers, chord reducer	Parallel CPU over large scans	Adding worker replicas
Query	FastAPI read endpoints, result tables, Redis cache	Low read latency, flat under data growth	Cache + read replicas

In a CRUD application these three concerns collapse into one request-response cycle, so a heavy analytical query blocks an ingestion write. Here a one-million-event job has no effect on ingest latency, because the two paths share only the database and touch different tables within it.

Out of scope

This is not a hospital management system. There is no patient portal, appointment booking, billing, clinical records UI, or real patient data. Patients exist only as opaque identifiers attached to events. Keeping this boundary firm is what makes the project finishable in fifteen days.

4. Cloud architecture

The system is organised in four tiers. Each communicates only with the tier adjacent to it, and no caller ever reaches a worker directly.

Tier	Components	Receives from	Sends to
1. Clients	Analyst and admin users (JWT), hospital event source (API key), load generator	-	API tier over HTTPS
2. API	FastAPI service, stateless, two replicas, 13 REST endpoints	Clients	Raw store, object storage, broker, cache
3. Processing	Redis broker, Celery worker pool of one to four replicas running map tasks and the chord reducer	Broker queue	Analytics store
4. Storage	PostgreSQL raw schema, PostgreSQL analytics schema, Redis result cache, object storage	API tier and workers	API tier

All three kinds of caller enter through the single stateless API tier. Writes land in the raw store; bulk CSV uploads are staged in object storage before loading. Job requests go onto the broker queue and are drained by the worker pool, which reads the raw store, performs map and reduce, and writes to the analytics store. Analytics reads are served from the analytics store through the Redis cache.

The defining property is a negative one: there is no direct path from the API tier to a worker. All work crosses the broker, which is what allows the request path and the processing path to scale and to fail independently.

Component	Responsibility	Why it is separate
FastAPI service	Validate, authenticate, persist raw events, enqueue jobs, serve results	Stateless, so replicas are interchangeable and it scales on request volume alone
Object storage	Hold bulk CSV uploads and exported reports	Large blobs do not belong in rows; keeps the database small
PostgreSQL (raw)	Immutable append-only event log, partitioned by month	Source of truth; every result can be regenerated from it
Redis (broker)	Job queue with late acknowledgement, plus shuffle keys	Decouples request from work; makes the system asynchronous
Celery worker pool	Run map tasks and the chord reducer	Scaling unit for processing; replica count is the throughput dial
PostgreSQL (analytics)	Derived metrics and the job ledger	Small, read-hot, rewritable; opposite characteristics from raw
Redis (cache)	60-second TTL on analytics responses	Keeps repeated reads off the database entirely

Two properties follow. The broker is the scaling boundary: everything above it scales on request volume, everything below on data volume, and the two are tuned independently. Raw and derived data are physically separate, with every derived row carrying the `job_id` that produced it.

5. Technology stack and justification

The selection rule: the lightest technology that genuinely demonstrates the required cloud behaviour and that a single student can account for line by line.

Layer	Choice	Alternatives considered	Why this one
API framework	FastAPI (Python 3.11)	Flask, Django REST, Express, Spring Boot	Async handling, Pydantic validation, and OpenAPI docs generated from the code, which satisfies requirement 10 at zero cost
Task queue / broker	Redis + Celery	RabbitMQ, SQS, Kafka, Spark	Celery `chord` is the exact map-then-reduce primitive needed; Redis doubles as the cache, so one service covers two components
Worker runtime	Celery workers in Docker	Threads in the API process, Lambda	Separate containers make horizontal scaling and the worker-kill failure visible; Lambda would hide them
Primary database	PostgreSQL 16	MongoDB, Cassandra, DynamoDB, SQLite	Justified in section 6
Cache	Redis, separate logical DB	Memcached, in-process LRU	Already present; yields a measurable cache-hit-ratio metric
Object storage	S3-compatible (MinIO locally)	Database BLOBs, local filesystem	CSV batches and exported reports belong in object storage; adds a genuine fourth cloud component
Auth	JWT + hashed API keys + RBAC	OAuth2 with external IdP, Firebase	Self-contained and demonstrable offline; no dependency on a third party's uptime
Migrations	Alembic	Raw SQL files, `create_all`	Versioned schema is a commit-worthy artefact
Load testing	Locust	JMeter, k6, Apache Bench	Scenarios in Python; HTML reports go straight into the report
Data generation	Faker + NumPy	Hand-written loops, public datasets	Real statistical distributions rather than uniform noise, which is what makes anomaly detection meaningful
Containerisation	Docker + Compose	Bare virtualenv, Kubernetes	Compose reproduces the full topology on a laptop; Kubernetes is wrong for a fifteen-day individual project
Deployment	Oracle Cloud Always Free VM, with Render as a costed fallback	AWS EC2, Railway, Fly.io, Azure for Students, Vercel	Serverless platforms are excluded outright: a Celery worker is a long-running process, and the failure and scaling demonstrations both require containers that can be counted and killed. Section 16 sets out the decision
Logging	structlog to JSON on stdout	Plain-string logging, ELK	JSON lines are greppable by correlation ID; ELK is a week of configuration
Metrics	Timing middleware + metrics endpoint	Prometheus + Grafana	Four numbers are needed, not a time-series platform

Why not Spark or Hadoop. The dataset tops out at one million rows and roughly 150 MB, which fits within a single worker's memory when streamed. Spark's distribution machinery would add overhead larger than the work itself at this scale, turning the project into an exercise in cluster configuration rather than pipeline design. Celery with a chord gives the identical logical structure (partition, map, shuffle, reduce) with real process-level parallelism across containers. This is a scale-appropriate choice, not an avoidance of distributed computing: the logical model demonstrated is the same.

6. Database selection and justification

Decision: PostgreSQL 16 as the system of record, Redis as cache and broker. Not MongoDB, not Cassandra.

The workload is specific: writes are append-only and never updated; processing reads are large sequential range scans filtered by time and department; query-time reads are tiny lookups against small pre-aggregated tables; the schema is fixed in advance; and aggregation (grouping, counting, averaging, windowing) is the core operation of the entire project.

Factor	PostgreSQL	MongoDB	Cassandra
Aggregation over time windows	Native window functions, `GROUP BY`, `percentile_cont`	Aggregation pipeline, verbose, weaker windowing	Very poor; built for key lookups
Fixed, known schema	Enforced at the database, catches generator bugs early	Schema-less; bugs surface late as bad analytics	Rigid but awkward to evolve
Range scans by timestamp	Excellent: B-tree index + native partitioning	Workable	Only along the clustering key
Join events to departments	Native	Awkward `$lookup` or denormalisation	Not supported
Transactional result writes	Full ACID; a reducer writes all rows or none	Multi-document transactions are a bolt-on	Eventually consistent by design
Operational cost for one student	Managed free tier everywhere, `psql` for live demonstration	Comparable	High; a cluster to operate

The deciding argument is that **this is an aggregation project.** The dominant operation is grouping millions of rows by time and category and computing statistics over them, which is precisely the workload relational engines have been optimised for. A document store would mean giving up window functions and joins in exchange for schema flexibility the project has no use for. Redis, a NoSQL key-value store, is used for the two jobs it is strictly better at: shuffle storage, where intermediate output is hundreds of short-lived keys read once and discarded, and result caching with a 60-second TTL. The accurate framing is therefore not SQL versus NoSQL, but relational for durable aggregated state and key-value for ephemeral hot state.

Requirement 13 is enforced as two logical schemas in one database. `raw.*` is append-only, never mutated, partitioned by month, and the immutable source of truth; if every result table were dropped, all analytics could be regenerated from it. `analytics.*` is derived, rewritable, small, and indexed for fast reads, and every row carries the `job_id` that produced it, giving complete traceability from any reported number back to the run that computed it.

7. Data model

Eight tables across two schemas: compact enough to be held in full by one developer, rich enough to model the domain.

`raw.hospital_events` is the central fact table, partitioned by month on `event_time`.

Column	Type	Notes
`event_id`	BIGSERIAL PK	
`patient_ref`	VARCHAR(32)	Opaque pseudonymous ID, never a real identity
`event_type`	ENUM	admission, discharge, transfer, icu_admit, icu_discharge, surgery
`department_id`	INT FK	References `raw.departments`
`event_time`	TIMESTAMPTZ	Partition key, B-tree indexed
`severity`	SMALLINT	1-5 triage score
`admission_source`	ENUM	emergency, referral, transfer, scheduled
`bed_id`	VARCHAR(16) NULL	Null for non-occupancy events
`length_of_stay_hours`	NUMERIC NULL	Populated on discharge events only
`ingested_at`	TIMESTAMPTZ	Server-stamped, never client-supplied
`correlation_id`	UUID	Threads this row through every log line
`batch_id`	UUID NULL	Set when loaded via bulk CSV rather than single POST

The three supporting raw tables are **raw.departments** (8 rows: ID, name, type, bed_capacity, staff_count, joined at map time), **raw.beds** (bed_id, department_id, bed_type, so occupancy is a percentage of real capacity rather than a raw count), and **raw.ingest_batches** (one row per bulk upload: batch_id, source_uri, row_count, uploaded_by, status, connecting object storage to the relational model).

analytics.admission_metrics has grain one row per (date, department).

Column	Type
metric_date, department_id	Composite PK
admission_count, discharge_count, transfer_count	INT
avg_severity	NUMERIC(3,2)
avg_length_of_stay_hours	NUMERIC(6,2)
peak_hour	SMALLINT
source_breakdown	JSONB, counts per admission source
job_id	UUID FK, lineage

analytics.resource_metrics has grain one row per (date, hour, department) and holds occupied_beds, bed_capacity, occupancy_rate, icu_occupancy_rate, staff_to_patient_ratio, job_id. Hourly grain is deliberate: occupancy is meaningless as a daily average, because a ward can run at 100% all morning and 40% all night.

analytics.anomaly_windows holds one row per detected anomaly: window_start, window_end, department_id, metric_name, observed_value, expected_value, z_score, severity_label, job_id.

analytics.analysis_jobs is the job ledger: job_id, job_type, status (queued/running/completed/failed/partial), params JSONB, requested_by, timestamps, duration_seconds, events_processed, partitions_total, partitions_completed, error_detail.

Design decisions

1. **Partitioning by month.** Partition pruning turns a March query into a scan of one partition instead of twelve, and makes the 10k/100k/1M comparison cleaner.
2. **Composite primary keys on result tables.** They make reducer writes idempotent via `INSERT ... ON CONFLICT DO UPDATE`, so re-running a job overwrites rather than duplicates. This is what makes the failure-recovery experiment safe to run live.
3. **JSONB for source_breakdown only.** A genuinely variable-shaped field goes in JSONB; everything fixed stays in typed columns.
4. **job_id on every derived row.** Any number in any API response traces to the job that computed it, with its parameters and timing.
5. **No patient table.** Patients are opaque references, a deliberate privacy-preserving choice appropriate to the healthcare domain.

8. Synthetic dataset generation

A uniform-random dataset would make every day look identical and the anomaly detector would find nothing, so the generator deserves real design attention.

Tier	Events	Period simulated	Approx. size	Purpose
Small	10,000	1 month	~2 MB	Development loop, unit tests, fast iteration
Medium	100,000	6 months	~18 MB	Realistic mid-size run
Large	1,000,000	3 years	~160 MB	Scalability proof, the headline benchmark

A clean 10x ladder makes the processing-time comparison easy to read. The generator uses NumPy distributions rather than uniform randomness, so the data carries genuine structure: **Poisson arrivals** with an hourly rate parameter peaking 10:00-12:00 and 18:00-21:00; **weekly seasonality**, with elective and surgical admissions dropping 60% at weekends; **annual seasonality**, a sinusoidal winter surge of about 30% in respiratory and general medicine; **log-normal length of stay**, short for most patients with a long tail, so mean and median differ; **severity correlation**, with higher triage severity raising ICU probability and stay length; and **department-specific profiles**, with Emergency hot and short, ICU small and long, Surgery weekday-heavy.

Injected anomalies and ground truth

Known anomalies are injected at known timestamps and recorded in a ground-truth file. The detector is evaluated against that file, which turns a subjective feature into a measurable one.

Injected event	Shape	What the detector should find
Mass-casualty incident	4x emergency admissions for 6 hours	A critical anomaly window, z-score above 4
Seasonal flu surge	+80% general medicine for 3 weeks	A sustained moderate/high window
ICU capacity crisis	ICU occupancy above 95% for 48 hours	A critical resource anomaly
Equipment outage	Surgical admissions near zero for 2 days	A negative anomaly: unusual low workload
Weekend staffing gap	Staff-to-patient ratio doubles on 3 weekends	Recurring moderate anomalies

The last row matters: the problem statement says unusual workload periods, not unusually high periods, so the detector flags deviations in both directions.

```
python -m scripts.generate --events 1000000 --start 2023-01-01 --end 2025-12-31 \
       --seed 42 --out data/large.csv --anomalies data/large_truth.json
```

A fixed seed means the same command always produces the same dataset, so all benchmark figures are reproducible and independently verifiable. The CSV is uploaded to object storage and bulk-loaded via Postgres `COPY`, roughly 50x faster than row-by-row inserts.

9. The analytics pipeline

This is the core of the project: a genuine MapReduce implemented with Celery primitives rather than a cluster framework.

Stage 1, partition. A coordinator divides the requested range into independent partitions, one per (day x department) pair. Six months across 8 departments gives 1,448 partitions, each written to `analysis_jobs.partitions_total` and dispatched as its own Celery task. The key choice is deliberate: day x department produces slices of roughly equal size, avoiding stragglers, the classic MapReduce failure mode where one oversized partition holds up the whole job. Partitioning by patient would produce wildly uneven slices.

Stage 2, map. Each task handles exactly one partition and never looks outside it, streaming its slice with a server-side cursor so memory stays constant regardless of partition size:

```
SELECT * FROM raw.hospital_events
WHERE event_time >= %s AND event_time < %s AND department_id = %s
```

For each row it emits a key of `(date, department_id)` and a partial accumulator holding counts, severity and length-of-stay sums, an hour histogram, source counts and a bed-occupancy delta. Map tasks are **pure and stateless**: they read raw data, emit values, and write nothing to the result tables. That purity is what makes them safely retryable, which is what makes the failure experiment work.

Stage 3, shuffle. Intermediate values are written to Redis under `job:{job_id}:key:{date}:{dept}`, so grouping by reducer key happens implicitly through the key naming. The shuffle set is small, a few thousand short-lived keys, because map tasks pre-aggregate within their own partition before emitting. This is the equivalent of a Hadoop combiner; the standard term is map-side combining.

Stage 4, reduce. Celery's `chord` fires the reducer only once every map task in the group has acknowledged completion. This is the synchronisation barrier and the single most important line of code in the project:

```
chord(map_tasks)(reduce_task.s(job_id))
```

The reducer folds the partial accumulators per key and computes what cannot be computed partially: admission counts, averages, peak hour and source breakdown; hourly occupancy, reconstructed by a running sum over bed-occupancy deltas divided by department capacity; and anomaly detection, a rolling 7-day mean and standard deviation per department, where any window deviating by more than 2.5 standard deviations in either direction is written to `anomaly_windows` with its z-score.

Stage 5, persist. Results are written with `INSERT ... ON CONFLICT DO UPDATE`, making the whole job idempotent. The job row flips to `completed` with its duration and event count. Partial failures mark the job `partial` rather than `failed`, with surviving partitions still written.

Why this is really MapReduce

MapReduce concept	Our implementation
Input split	Day x department partition
Mapper	Celery map task, pure function, one partition
Combiner	Pre-aggregation inside the map task before emit
Partitioner / shuffle	Redis keys namespaced by reducer key
Barrier	Celery chord header completion
Reducer	Chord callback folding accumulators per key
Output	`analytics.*` tables
Speculative re-execution	Celery retry with `acks_late` on worker loss

The difference from Hadoop is the scheduler and the storage layer, not the computational model.

10. REST API design

Thirteen endpoints against a minimum of five. Each serves a distinct function; none exists solely to increase the count.

#	Method	Path	Auth	Purpose
1	POST	`/api/v1/auth/register`	Public	Create an account; role assigned by admin
2	POST	`/api/v1/auth/login`	Public	Exchange credentials for an access + refresh JWT pair
3	POST	`/api/v1/auth/refresh`	Refresh token	Rotate an expired access token
4	POST	`/api/v1/events`	API key	Ingest a single hospital event
5	POST	`/api/v1/events/batch`	API key	Bulk ingest from an object-storage URI; returns a batch ID
6	GET	`/api/v1/events`	JWT, any role	Paginated raw event query with filters
7	POST	`/api/v1/jobs`	JWT, analyst+	Submit an analytics job; returns `202` with a job ID
8	GET	`/api/v1/jobs/{id}`	JWT, analyst+	Job status, progress, duration
9	GET	`/api/v1/jobs`	JWT, analyst+	List jobs with status filter

#	Method	Path	Auth	Purpose
10	GET	`/api/v1/analytics/admissions`	JWT, any role	Admission patterns by date range and department
11	GET	`/api/v1/analytics/resources`	JWT, any role	Occupancy and staffing utilisation, hourly grain
12	GET	`/api/v1/analytics/anomalies`	JWT, any role	Unusual workload windows, filterable by severity
13	GET	`/api/v1/metrics`	JWT, admin	Operational metrics: latencies, throughput, cache hit ratio

Plus `GET /health` (unauthenticated, for platform health checks) and `GET /docs` (auto-generated Swagger UI).

The asynchronous trigger is the endpoint that carries the architecture:

```
POST /api/v1/jobs
{ "job_type": "admission_patterns", "start_date": "2025-03-01",
  "end_date": "2025-03-31", "departments": [1, 2, 3] }

202 Accepted
{ "job_id": "8f3c...", "status": "queued", "partitions_total": 93,
  "poll_url": "/api/v1/jobs/8f3c..." }
```

The `202` rather than `200` is the single most important design decision in the API. It is the HTTP-level expression of the whole architecture: the server has accepted responsibility for the work but has not done it. A CRUD application has no reason to ever return `202`. The paired progress endpoint, `GET /api/v1/jobs/{id}`, makes asynchrony visible on screen as `partitions_completed` climbs.

The payoff endpoint is `GET /api/v1/analytics/anomalies`, which returns detected windows with observed value, expected value, z-score and severity. Its output is checked against the ground-truth file to confirm the pipeline located the injected incidents.

Conventions across all endpoints

- Versioned under `/api/v1/` so the path documents an upgrade strategy.
- Cursor pagination on every list endpoint (`limit`, `cursor`), never offset, because offset pagination degrades badly at a million rows.
- Consistent error envelope `{ error: { code, message, correlation_id } }`, so any failed request can be located in the logs immediately.
- Correct status codes: `201` create, `202` accepted-for-processing, `400` validation, `401` unauthenticated, `403` wrong role, `404`, `409` duplicate, `422` semantic validation, `429` rate-limited.
- Rate limiting on ingest endpoints via a Redis token bucket.

11. Authentication and authorization

Two credential types, because there are genuinely two kinds of caller: humans who log in, and machines that stream events.

Humans use JWT with refresh rotation. Login returns a 15-minute access token and a 7-day refresh token. Access tokens are signed HS256 and carry `sub`, `role`, `exp`, `iat`, `jti`. Passwords are hashed with bcrypt at cost factor 12: never stored, never logged, never returned. Refresh tokens are stored hashed in Redis and rotated on use; if an already-used refresh token reappears, that indicates theft and the whole token family is revoked. This exceeds the stated requirement at a cost of roughly thirty lines.

Machines use hashed API keys. Ingest clients authenticate with `X-API-Key`. Keys are 32 bytes of `secrets.token_urlsafe`, shown once at creation and stored only as a SHA-256 hash, each with a scope and a rate limit, and every ingested event records which key wrote it. The reasoning: a hospital information system

pushing events continuously cannot perform an interactive login, and a long-lived JWT would be an unrevocable credential with a user identity attached. An API key is independently revocable and rate-limited and carries no user context, which is correct, because no user is involved.

Capability	`viewer`	`analyst`	`admin`	API key
Read analytics results	Yes	Yes	Yes	No
Read raw events	Yes	Yes	Yes	No
Submit analytics job	No	Yes	Yes	No
Cancel / re-run a job	No	Own jobs	Any job	No
Ingest events	No	No	Yes	Yes
Trigger bulk load	No	No	Yes	Yes
View operational metrics	No	No	Yes	No
Manage users and keys	No	No	Yes	No

Enforcement is a FastAPI dependency rather than an `if` statement inside each handler:

```
@router.post("/jobs", dependencies=[Depends(require_role(Role.ANALYST))])
```

Centralising it means authorisation cannot be forgotten on a new endpoint, and it is the mechanism by which privilege escalation is prevented. The live demonstration is four commands: `viewer` submits a job and receives `403`; `analyst` makes the identical call and receives `202`; an ingest call with a JWT instead of an API key receives `401`; and a hashed key is shown in the database with no plaintext anywhere in the system.

Beyond the requirement: secrets come from environment variables with `.env` git-ignored, CORS is restricted to known origins, SQL goes exclusively through SQLAlchemy parameterised queries with no string interpolation anywhere, and Pydantic validates every request body.

12. Logging and observability

Being able to trace one event through five services by a single ID is what allows the pipeline's behaviour to be demonstrated rather than asserted.

Logs are structured JSON on stdout, one object per line, via structlog:

```
{ "ts": "2026-10-12T09:14:22.481Z", "level": "info", "service": "worker-2",
  "event": "map_task.completed", "correlation_id": "7b2e...",
  "job_id": "8f3c...", "partition": "2025-03-14/cardiology",
  "rows_read": 1847, "duration_ms": 312 }
```

JSON rather than prose because it is machine-filterable: one `jq` filter on a correlation ID reconstructs an entire request path. Stdout rather than files because containers are ephemeral; the platform collects stdout, and writing log files inside a container is an anti-pattern.

Correlation IDs. A UUID is generated at the API edge (or taken from an inbound `X-Correlation-ID` header), attached to the request context, stored on the event row, passed into the Celery task payload, and logged by every worker that touches it. One ID therefore spans API, database, broker and worker. This is the single feature that makes the mandatory single-record demonstration straightforward.

Stage	Events logged	Level
Request edge	Method, path, status, duration, user or key, correlation ID	info
Auth	Login success and failure with reason, token refresh, role denial	info / warning
Ingest	Event accepted with ID, validation rejection with field	info / warning

Stage	Events logged	Level
Job lifecycle	Job ID, type, parameters, partition count, final status and duration	info
Map and reduce	Partition, rows read, retry count, keys folded, rows written, durations	info
Shuffle	Keys written, total intermediate size	debug
Failures	Exception type, message, stack trace, retry decision	error
Worker lifecycle	Startup, shutdown, task acknowledgement, task loss	warning

Levels are used deliberately: `debug` for shuffle detail (off in production), `info` for the normal path, `warning` for handled but notable events such as a retry or a denied role, and `error` strictly for unhandled failures. Keeping `error` rare is what makes it useful, so the log stays clean during normal operation and produces a single precise line at the moment a worker is killed.

Never logged: passwords, raw API keys, JWTs, and full request bodies containing patient references. A log line records that patient `p_48213` was admitted; it never records anything that would identify a person.

13. Metrics

The requirement is two measurable metrics. Four are instrumented, because they are cheap once the timing middleware exists and together they describe the system rather than two isolated numbers.

Metric	Definition	How measured	Why it matters
M1 Job processing time	Wall-clock seconds from job dequeue to result commit	`started_at` / `finished_at` on the job row	The headline data-intensive metric; basis of the three-dataset comparison
M2 Ingestion throughput	Events written per second, single and bulk	Rows / elapsed, recorded per batch	Shows the write path scales independently of analytics
M3 API latency p50/p95/p99	Per-endpoint response time distribution	Middleware timer, percentiles in a Redis sorted set	Proves query latency stays flat as raw volume grows 100x
M4 Cache hit ratio	Cache hits / total analytics reads	Redis counters	Justifies the cache as a real component rather than decoration

The central experiment is M1 across the three dataset sizes, at one and at four workers:

Dataset	Events	Partitions	1 worker	4 workers	Speed-up	Throughput (events/s)
Small	10,000	31				
Medium	100,000	186				
Large	1,000,000	1,095				

Two findings are expected, and either outcome can be explained. Processing time should scale roughly linearly with event count; if it scales worse, the cause is almost certainly a missing index or per-partition overhead dominating at small sizes, and the report will identify which with supporting evidence. Four workers should *not* give a 4x speed-up: expect 2.5-3x, the gap being Amdahl's law, since the reduce stage is serial and the chord barrier means the job is only as fast as its slowest partition. Predicting this in advance and confirming it experimentally is the result we expect to report.

A secondary experiment measures p95 on `GET /api/v1/analytics/admissions` at all three sizes. Latency should barely move, because the endpoint reads pre-aggregated tables whose size depends on date range and department count rather than raw event volume. That flat line is the clearest single piece of evidence that the architecture does what it claims. Results surface through `GET /api/v1/metrics` for the admin role as well as in the report, so any figure can be verified live.

14. Load and scalability testing

Three workload levels are required; four are run, adding a breaking-point test, because finding where the system actually fails is a more useful result than three runs that all pass.

Level	Concurrent users	Spawn rate	Duration	Run against	Expected outcome
Low	10	2/s	5 min	Local and cloud	All green; establishes the baseline
Medium	100	10/s	5 min	Local and cloud	p95 rises, error rate stays at zero
High	500	25/s	5 min	Local only	Latency degrades visibly; some `429`s from rate limiting
Breaking point	1,000	50/s	until failure	Local only	Identifies the actual bottleneck

Where the high levels run depends on the platform chosen in section 16. On the Oracle virtual machine, with 12 GB of memory and no per-service connection caps, all four levels run against the deployed system and one set of numbers covers everything. On the managed-service fallback, the free instances cap Postgres at 100 connections and Key Value at 50 on small shared compute, so driving 500 or 1,000 users at them would measure the hosting plan's ceiling rather than the application's. In that case the high and breaking-point runs execute against Docker Compose on a machine whose specification is recorded, and the 10- and 100-user runs are repeated in cloud for comparison. Whichever applies is stated plainly in the results: a load test whose limiting factor is understood is worth more than a larger number that is not.

Locust runs a weighted mix rather than hammering one endpoint, because a realistic mix is what reveals interaction effects: 50% read analytics results, 25% ingest single events, 15% poll job status, 7% query raw events with filters, 3% submit a new job. At each level we record requests per second sustained, p50/p95/p99 latency per endpoint, error rate by status code, and database connection pool saturation.

Separately, the scalability experiment runs the identical 1,000,000-event job at one, two and four worker replicas and records completion time. This is a different claim from the load test: load testing measures the API under concurrent users, scalability testing measures the processing tier under added capacity. Keeping the two conceptually separate is worth doing, since many submissions conflate them. It runs locally, where the replica count is a single Compose flag.

Prediction. The bottleneck at the high level should be the database connection pool rather than CPU: FastAPI will accept 500 concurrent requests, which then queue for one of twenty pooled connections. The remedy, raising pool size, adding `pgbouncer`, or caching more aggressively, is itself a finding worth writing up. A load test that identifies a bottleneck and explains the fix is more useful than one reporting no issues.

Locust produces an HTML report per run with latency distributions and a failure table. Four reports go into the repository under `/benchmarks`, with a comparison table and a short analysis in the final report.

15. Controlled failure scenario

One controlled failure is required; three are implemented, and the first is demonstrated live, because a failure that can be induced on command is stronger evidence than one described in prose.

Primary: worker death mid-job. Start a 1,000,000-event job across four workers, let it reach roughly 40% partition completion, then `docker kill hospital-worker-2`.

Celery is configured with `acks_late=True`, so a task is acknowledged to the broker only after it completes, not when it is received. Worker 2 had partitions in flight that were never acknowledged; when the broker's visibility timeout expires, those tasks become visible again and the surviving three workers pick them up. The chord barrier simply waits longer, and the job completes with the correct result and a longer duration.

Correctness is preserved because map tasks are pure functions over an immutable input partition: they read raw data and emit values, writing nothing, so re-executing one produces identical output. The reducer writes with `ON`

`CONFLICT DO UPDATE`, so even a duplicated reduce is idempotent. This combination of pure maps and idempotent writes is what allows the system to survive the loss of a worker.

Evidence captured: the `docker kill` timestamp; the warning line recording task loss; the redelivery lines on surviving workers; the job row showing `completed` with an elevated duration; and a row-count comparison proving the result is identical to the no-failure run. That last check proves correctness rather than merely liveness.

Secondary: database connection loss. Stop Postgres mid-job. Tasks fail on connection error, Celery retries with exponential backoff (1s, 2s, 4s, 8s, capped at five attempts), and the job is marked `failed` with a structured `error_detail` if the database never returns. Restore it within the retry window and the job completes. This demonstrates that transient failures (retry) are distinguished from permanent ones (fail cleanly and record why) rather than retrying forever.

Tertiary: broker unavailability. Stop Redis. Job submission returns `503 Service Unavailable` with a clear message rather than hanging or returning a misleading success. Critically, ingestion keeps working, because the write path does not touch the broker: the analytics tier is down but data is still being captured and nothing is lost. For a hospital system that is the correct failure mode, since analytics can wait but clinical events cannot be lost. In one sentence, the recovery mechanism across all three cases is this: pure, retryable map tasks over immutable input, plus idempotent result writes, plus late acknowledgement, mean any worker can die at any point and the job still produces the correct answer, only slower.

16. Deployment

Four containers, built from two Dockerfiles:

Container	Image	Role	Replicas
`api`	app image, uvicorn entrypoint	Serves REST	1 locally, 2 in cloud
`worker`	same app image, celery entrypoint	Runs map/reduce tasks	1-4, varied for experiments
`postgres`	postgres:16-alpine	System of record	1 (managed in cloud)
`redis`	redis:7-alpine	Broker, shuffle, cache	1 (managed in cloud)

The API and worker share one image and differ only in entrypoint. This is deliberate: identical code in both, no drift between what the API validates and what the worker processes, and one build to maintain. Local development is `docker compose up` and nothing else; MinIO is added locally to stand in for S3.

Repository layout

```
hospital-event-analytics/
|-- app/
|   |-- api/v1/          routes: auth, events, jobs, analytics, metrics
|   |-- core/            config, security, logging, dependencies
|   |-- models/          SQLAlchemy models
|   |-- schemas/         Pydantic request/response schemas
|   |-- pipeline/        partition.py, map_tasks.py, shuffle.py, reduce.py
|   |-- services/        business logic, cache, storage
|   `-- main.py
|-- scripts/             generate.py, benchmark.py, load_csv.py
|-- loadtests/           locustfile.py
|-- migrations/          alembic
|-- benchmarks/          results, Locust HTML reports, failure evidence
|-- docs/                architecture diagram, API docs, report
|-- tests/
|-- docker-compose.yml   Dockerfile   render.yaml   README.md
```

The `pipeline/` package is deliberately separate from `api/`, so the architecture is legible from the folder names before any code is read.

The `pipeline/` package is deliberately separate from `api/`, so the architecture is legible from the folder names before any code is read.

Choosing the hosting platform

Serverless platforms are excluded before cost is considered. A Celery worker is a long-running process that polls a broker, which has nowhere to live on a function-per-request platform, and the controlled failure experiment and the scaling experiment both depend on containers that can be counted, scaled and killed. That rules out Vercel, Netlify and comparable services regardless of their free tiers.

Two viable options remain. The choice is settled early in the build rather than at deployment time, because one of them depends on an account approval outside our control.

	Primary: Oracle Cloud Always Free VM	**Fallback: Render managed services**
Shape	One virtual machine, 2 OCPU and 12 GB RAM, running the full Compose topology	Separate managed services: web service, background worker, Postgres, Key Value
Cost	None	The web service and both datastores run on free instances; background workers are not offered on a free instance type, so the worker runs on the lowest paid plan for the project month
Scaling demonstration	`docker compose up --scale worker=4`, with containers appearing live	Replica count adjusted from the platform dashboard
Experiments	12 GB of memory is enough to run the 1,000,000-event benchmark and the full load-test range in the deployed environment	Free instance limits force the high-load runs to be executed locally
Risk	Account approval can be slow or can fail on payment verification, so it is requested early and not relied upon until confirmed	Requires a payment method for the worker instance

The primary option is preferred because it removes the main compromise in section 14: the high-concurrency runs execute against the deployed system rather than a laptop, so one set of numbers covers both. The fair objection is that a single virtual machine is less visibly distributed than a set of managed services; the answer is that the deployed topology is identical either way, with separate containers for the API, the worker pool and both datastores, and the scaling unit is still the worker replica count. The repository carries a `render.yaml` blueprint regardless, so the managed-service deployment stays one command away and the blueprint documents the intended production shape.

Hosting constraints and how they are handled

If the fallback is used, its free instance types carry limits that are planned for rather than discovered late. The paid worker instance is covered above.

Constraint	**Effect on this project**	**How it is handled**
Free Postgres expires 30 days after creation and is deleted after a grace period	A database provisioned too early could expire before evaluation	It is created close to deployment, and the schema is reproducible from Alembic migrations in seconds
Free Key Value is in-memory only and may restart at any time	A restart mid-job loses queued tasks and shuffle state	Jobs are re-runnable and result writes idempotent, so a lost job is re-submitted rather than corrupting data
Free web services spin down after 15 minutes idle	A cold start during evaluation looks like a broken deployment	A scheduled ping every ten minutes keeps the service warm
Free Postgres allows 100 connections, free Key Value 50	High-concurrency load tests would measure the plan, not the application	High-load runs execute locally; see section 14

Configuration follows twelve-factor on either platform: all settings via environment variables read once into a Pydantic `Settings` object at startup, no secret ever committed, and `.env.example` shipped with placeholders so the repository is runnable by anyone.

Fallback. The demonstration must not depend on the hosted environment behaving on the day. A fully working `docker compose up` on the laptop is maintained as a hot standby throughout, running an architecture identical to the deployed one. If the deployment misbehaves, the switch takes under a minute, and it will be rehearsed in advance rather than attempted for the first time under evaluation.

17. Demonstration plan

The mandatory demonstration asks for API execution, database and storage state, logs and metrics, an explanation of the cloud architecture, and a trace of one sample record through the pipeline. The run below covers all of it in roughly twelve minutes, with four panes arranged in advance: a terminal with curl, a connected `psql` session, a `docker logs -f` tail of the workers, and a browser on `/docs`.

Min	What	What it proves
0-1	Present the cloud architecture: components, tiers and data flow	Requirement: explain the cloud architecture
1-2	Log in as `viewer`, submit a job, get `403`; log in as `analyst`, get `202`	Auth and RBAC
2-3	`POST` one event with a visible correlation ID; show the row in `psql`	Ingest and raw storage
3-5	Submit a job over the window containing that event; poll and watch `partitions_completed` climb	Asynchrony made visible
5-7	**The trace.** Grep the correlation ID in the worker logs: partition assigned, rows read, intermediate emitted, reduce folded, rows written	The mandatory single-record demonstration
7-8	`SELECT` the result rows in `psql`; point out the `job_id` lineage column	Processed storage, separate from raw
8-9	`GET /api/v1/analytics/anomalies`; open the ground-truth file and show the injected incident was found	The analytics are correct
9-10	**Kill a worker mid-job.** Show the loss warning, the redelivery, the job completing	Controlled failure and recovery
10-11	Show the benchmark table across three dataset sizes and the worker-scaling result	Data-intensive requirement
11-12	`GET /api/v1/metrics` live; show the Locust report from the 500-user run	Metrics and load testing

The worker kill at minute 9 is the only point at which a failure is deliberately induced and handled in view of the evaluator; its timing will be rehearsed until exact.

Anticipated questions

Question	Answer
Why not Spark or Hadoop?	At 1M rows the distribution overhead would exceed the work; Celery chords give the identical map/shuffle/reduce model with real process parallelism.
Why SQL and not NoSQL?	The dominant operation is aggregation over time windows, which relational engines are built for; Redis is used where a key-value store is stronger.
How is this not CRUD?	A CRUD application never returns `202`. The write, processing and read paths are separate tiers that scale independently.
What if two jobs cover the same window?	Result writes use `ON CONFLICT DO UPDATE` keyed on (date, department), so the operation is idempotent.
How do you know the anomaly detection is correct?	Anomalies are injected at known timestamps with a ground-truth file; detection is scored against it, not judged by eye.
What is your bottleneck?	The database connection pool under high concurrency; it was measured, and commit 14 is the fix.
How would you scale to 100 million events?	Partition by week rather than day, move shuffle to object storage, switch to Spark, add read replicas for the query tier.

Question	Answer
Where would this break in production?	Single-region deployment, no dead-letter queue for permanently failing partitions, and the shuffle is memory-bound in Redis.

The last two are answered deliberately: the limitations are known and stated rather than omitted, each with a remedy.

For the live modification, the likely asks are a new filter on an endpoint, a fourth aggregate in the reducer, or a new RBAC role. Adding an aggregate touches exactly three places: the map emit, the reduce fold, and the response schema.

18. Risks and scope management

Risk	Likelihood	Impact	Mitigation
Celery chord behaves unexpectedly at scale	Medium	High; it is the core	Prototype the chord on the small dataset before anything depends on it; fallback is a polling reducer that checks partition completion itself
1M-row job exhausts worker memory	Medium	High	Server-side cursors from the start, never `fetchall()`; test at 1M early enough to leave time to fix
Free-tier cloud sleeps or is slow during demonstration	Medium	Medium	Keep-warm ping, plus a rehearsed local Compose fallback
Load testing reveals a serious flaw late	Medium	Medium	Buffer is reserved at the end of the build; a documented flaw with analysis remains a valid result. Dataset generation is similarly staged: 10k first, larger tiers generated overnight
Anomaly detector finds nothing of interest	Low	High for the demonstration	Anomalies are injected deliberately with ground truth, so this is largely designed out
Free Key Value restarts and loses queued tasks mid-demonstration	Low	Medium	Jobs are idempotent and re-runnable; the demonstration job is short, and the local stack is the standby
Paid worker instance not provisioned in time	Low	High	Provision and bill it alongside the deployment, not on the morning of the demonstration
Scope creep into hospital-management features	Medium	High	Section 3 states explicitly what is out of scope and is treated as binding

If we fall behind, scope is cut strictly in this order, and nothing in the list touches a stated requirement. The first four are treated as optional from the outset rather than as planned work: the GitHub Actions pipeline, useful discipline but not asked for; rate limiting and the token bucket, a production concern rather than an assessed one; object storage, replaced by direct CSV upload to the API; and refresh-token rotation, since a plain JWT pair satisfies the requirement. After those, in order: the 1,000-user breaking-point test, since three workload levels are what is required; resource-utilisation metrics, since admissions and anomalies still answer the problem statement; and last the Redis caching layer, which costs the cache-hit metric.

Never cut: the MapReduce pipeline, the three dataset sizes, the failure experiment, the architecture diagram, and the deployment. Those five constitute the core of the assigned specification.

This is the design document, written before the build. It becomes the final report once the work is done: the empty result tables in sections 13 and 14 are filled with measured numbers and screenshots are added throughout, so any divergence between what was designed and what was built stays visible.